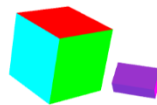


PA2 — Matrix Transformations and Perspective

Prepared by Kossaly Diana

Student ID: 241629

Student ID: 241629 | Projection: PERSPECTIVE | FOV: 60° | t: 13.8 | FPS: 60.0



Активация Windows
Чтобы активировать Windows, перейдите в раздел
"Параметры".

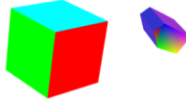
E1 — Original

Matrix order: $M = R_{\text{orbit}} \cdot T \cdot R_{\text{self}} \cdot S$

Prediction: The prism will spin around its own Y-axis while orbiting the cube at a radius of 2.5 units.

Result: In Screenshot 607, the prism is on the right side of the cube at $t = 13.8$ s, near the end of its 15-second orbit.

Explanation: glmatrix post-multiplies each transformation call, so the last call acts on a vertex first: $S \rightarrow R_{\text{self}} \rightarrow T \rightarrow R_{\text{orbit}}$. The prism is scaled and self-spun, translated away from the cube, then carried around the cube by the orbit rotation.



Активация Windows
Чтобы активировать Windows, перейдите в раздел
"Параметры".

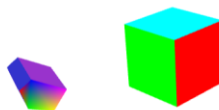
E1(a) — Swap orbit rotation and translation

Matrix order: $M = T \cdot R_{\text{orbit}} \cdot R_{\text{self}} \cdot S$

Prediction: The prism's center will stay at a fixed offset instead of orbiting the cube, although its orientation will continue to change.

Result: In Screenshot 606, the prism remains to the right of the cube at $t = 6.9$ s.

Explanation: The operations on a vertex are $S \rightarrow R_{\text{self}} \rightarrow R_{\text{orbit}} \rightarrow T$. Translation is applied last to the vertex, so the orbit rotation does not rotate the prism's offset around the cube.



Активация Windows
Чтобы активировать Windows, перейдите в раздел
"Параметры".

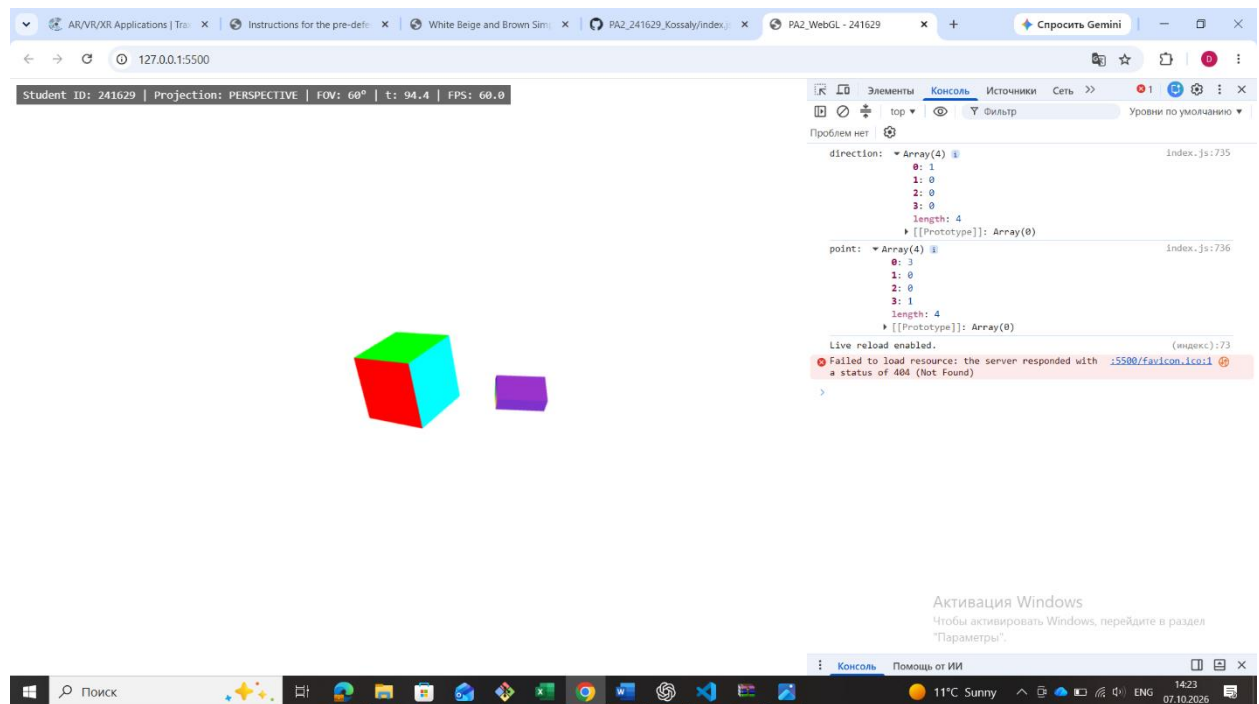
E1(b) — Swap translation and self-spin

Matrix order: $M = R_{\text{orbit}} \cdot R_{\text{self}} \cdot T \cdot S$

Prediction: The prism's position will change because the self-spin rotation also rotates its translation away from the cube.

Result: In Screenshot 609, at $t = 7.0$ s, the prism appears to the left and below the cube.

Explanation: The operations on a vertex are $S \rightarrow T \rightarrow R_{\text{self}} \rightarrow R_{\text{orbit}}$. Since translation happens before the rotations, both Y-axis rotations affect the prism's position. That couples its self-spin to its orbit.

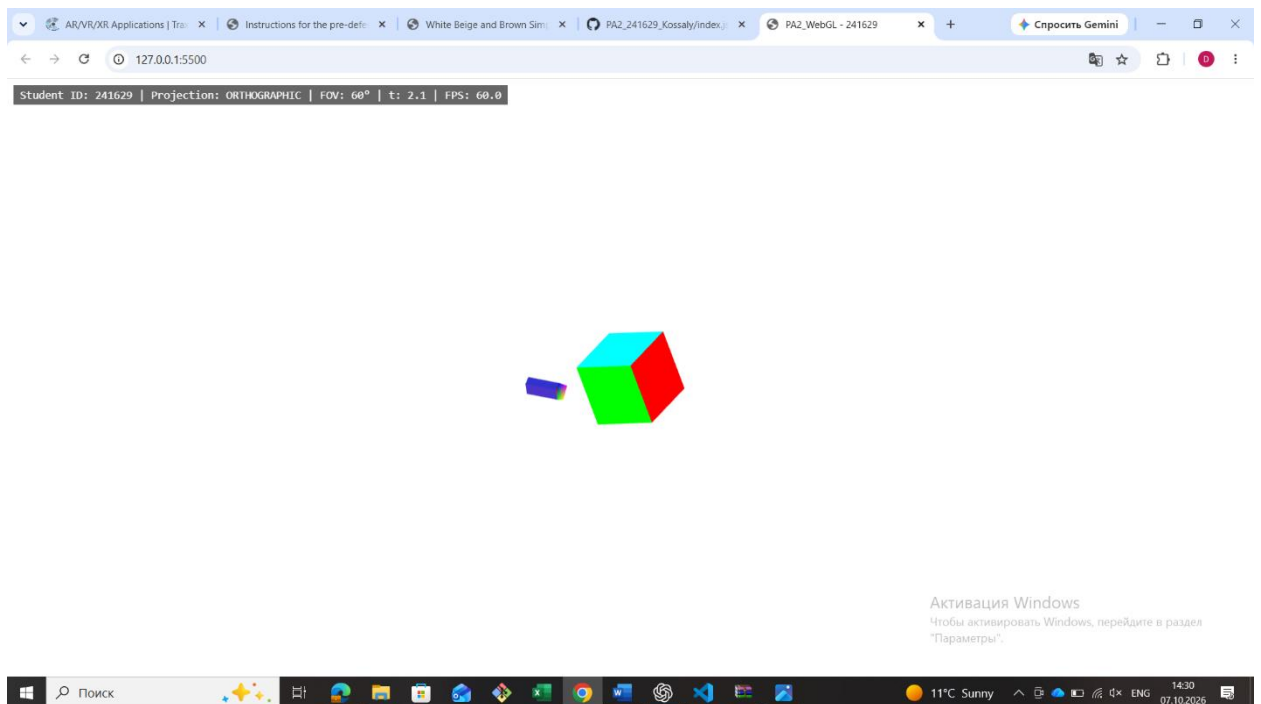
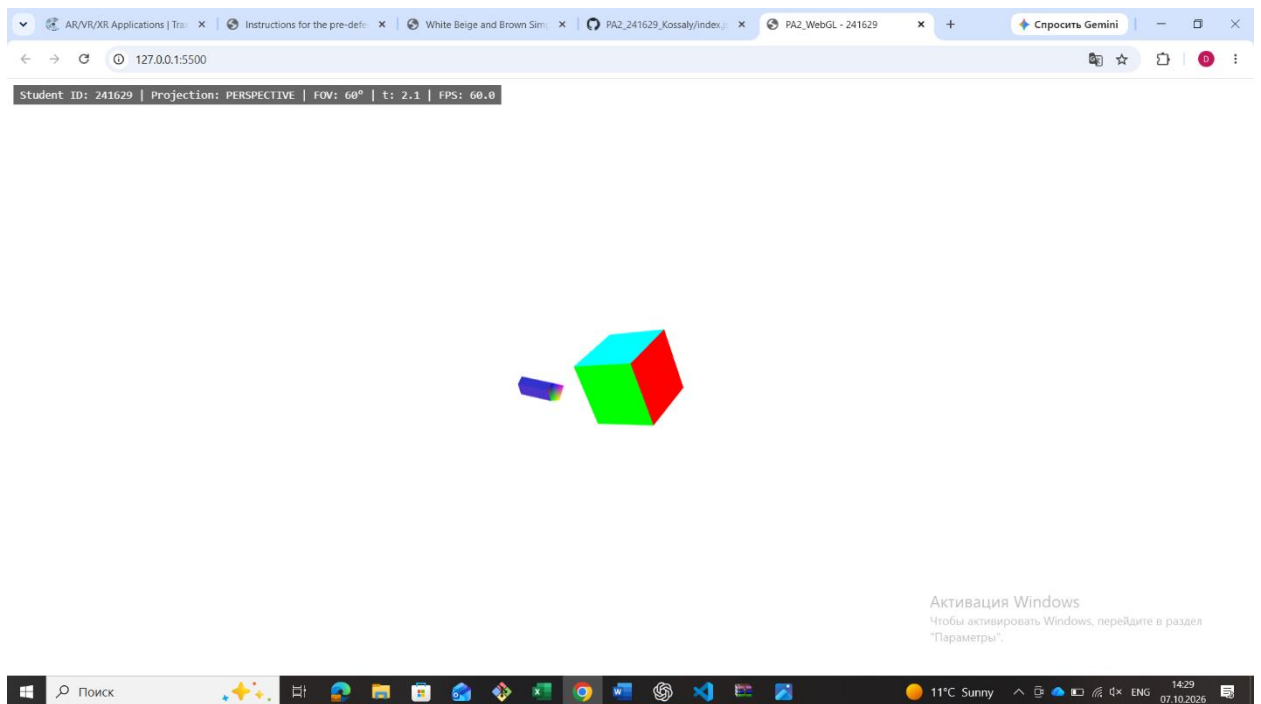


E2 — Correcting three statements from Chapter 4

(a) Rotation axis. The comment says that the second `mat4.rotate` rotates around X, but its axis argument is `[0, 1, 0]`, which specifies Y. The comment should say “Y-axis,” or the vector should be changed to `[1, 0, 0]` if X was intended.

(b) Clipping and culling. Geometry outside the view frustum is clipped against the clip volume. Face culling is a separate optional operation that discards front- or back-facing polygons according to their winding direction.

(c) Homogeneous coordinate w . The homogeneous coordinate is not a quaternion component. With a translation of $(2, 0, 0)$, the direction $[1, 0, 0, 0]$ remains unchanged, while the point $[1, 0, 0, 1]$ becomes $[3, 0, 0, 1]$. The console output in Screenshot 613 confirms this: `direction: [1, 0, 0, 0]; point: [3, 0, 0, 1]`. A direction has $w = 0$, so translation does not affect it; a point has $w = 1$, so it is translated.



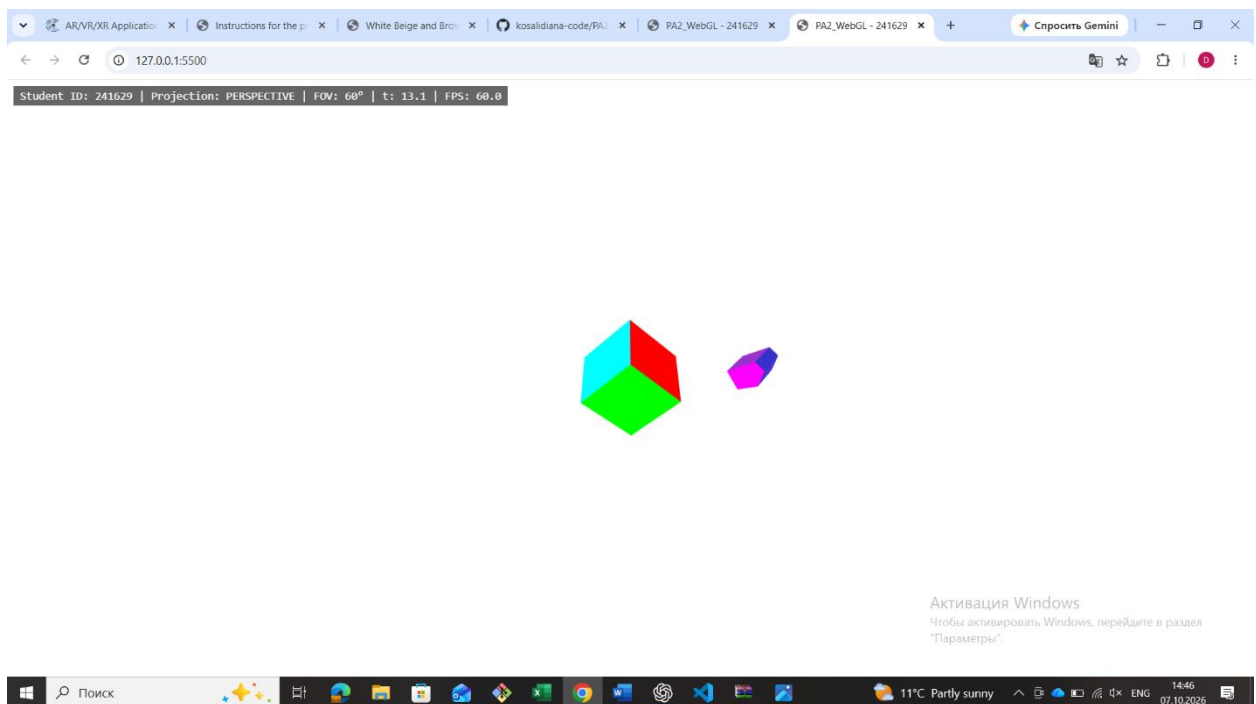
E3 — Perspective vs. orthographic projection

Prediction: In perspective projection, the nearer vertical edge will appear longer than the farther edge. In orthographic projection, the two parallel edges will have approximately the same length.

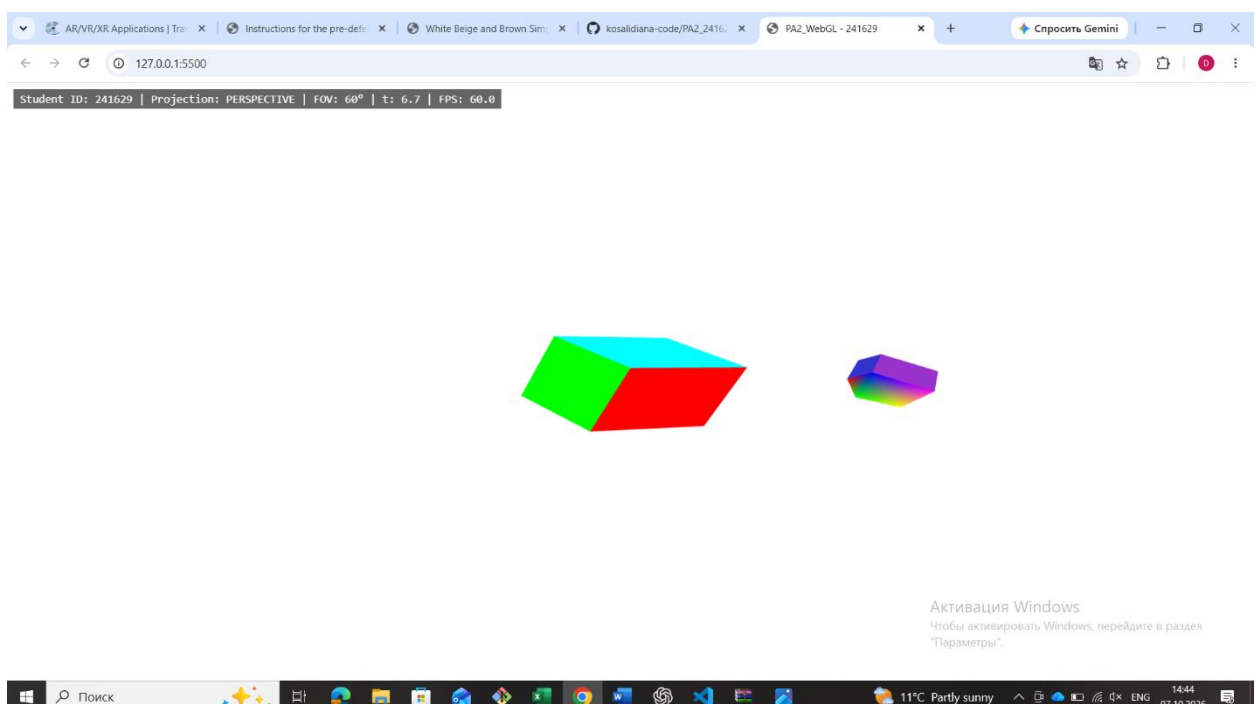
Result: At the same paused pose ($t = 2.1$ s), the perspective screenshot shows edge lengths of approximately 102 px and 96 px, giving a ratio of 1.06. In the orthographic screenshot, the corresponding edges are both approximately 93 px, giving a ratio of 1.00.

Explanation: Perspective projection divides clip-space coordinates by w , which depends on distance from the camera. This makes the farther edge appear smaller. Orthographic projection does not scale objects by distance, so parallel edges keep approximately the same on-screen

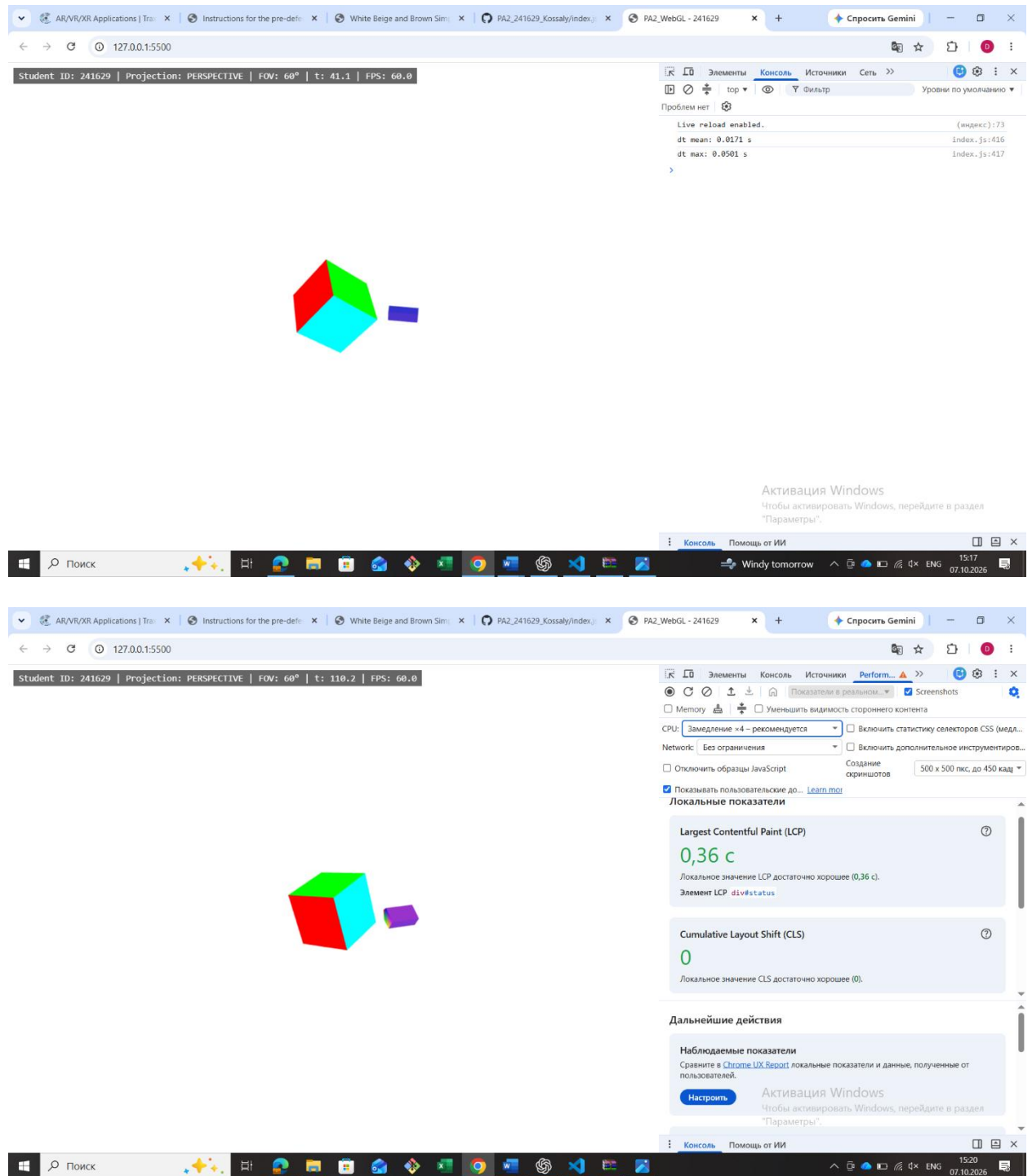
length. PA1's fixed depth offset resembles orthographic projection because it changes depth without making the farther object appear smaller.

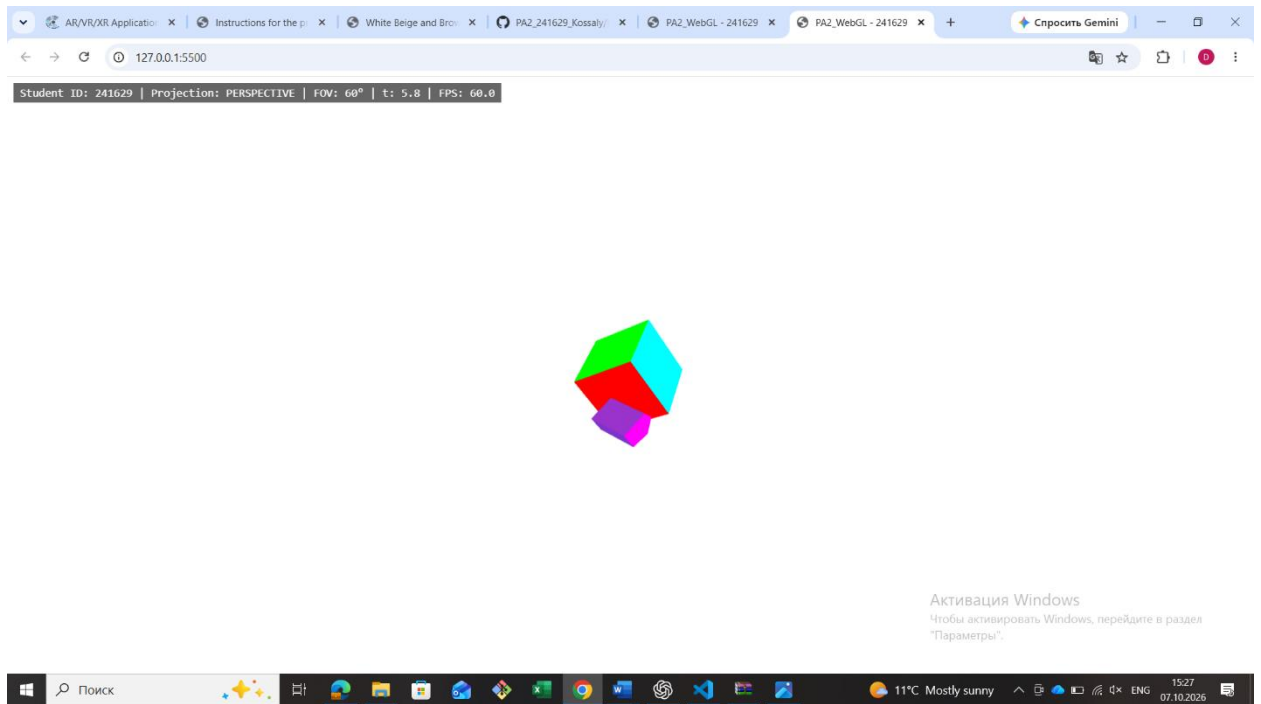


E4(a). The near clipping plane was temporarily set to 5.8. At approximately $t = 13.1$ s, the orbiting prism moved close enough to the camera that part of it was clipped. This demonstrates how the near plane removes geometry closer to the camera than the specified distance. For the final scene, the near plane was restored to 0.1 and the far plane remained 20.0. The eye is approximately 7.68 units from the origin. During the orbit, the distance from the eye to the prism's orbit center ranges from about 5.47 to 10.03 units. The prism's maximum vertex radius after scaling is about 0.58 units, giving a conservative eye-to-vertex range of approximately 4.89 to 10.61 units. The cube vertices are also within this range. Therefore, $near = 0.1$ and $far = 20.0$ safely include the scene throughout the orbit.

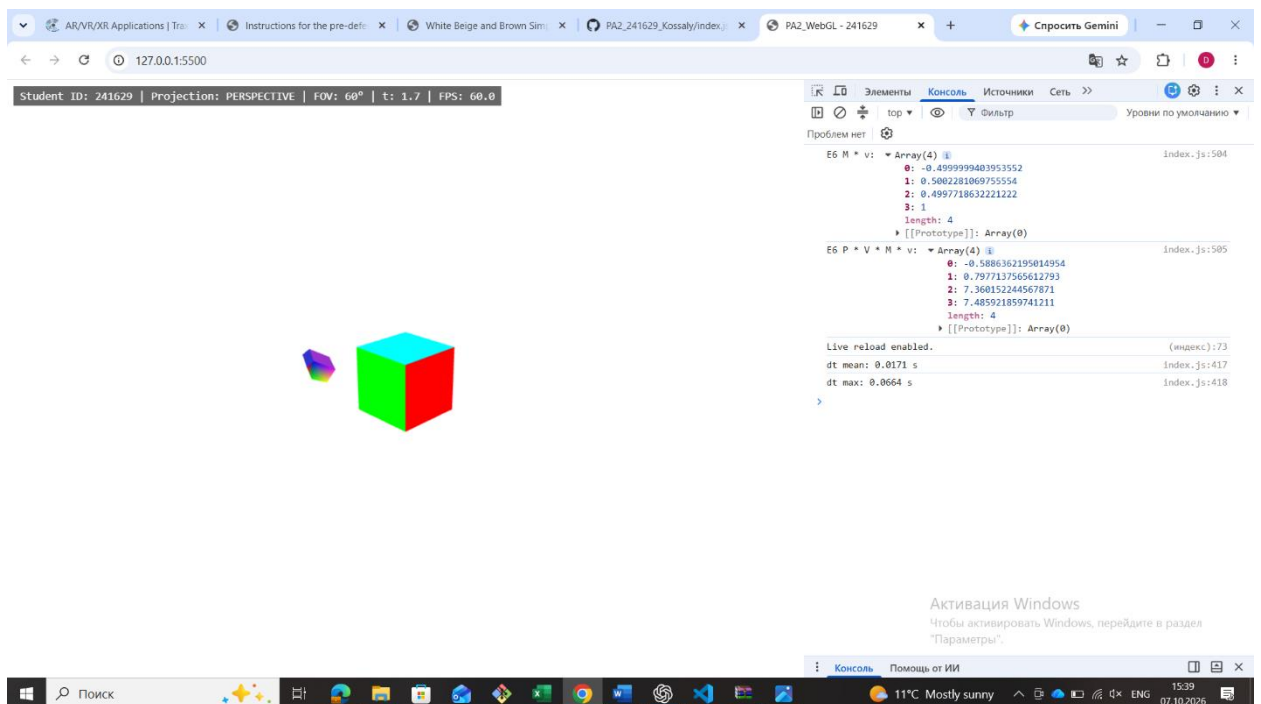


E4(b). The projection aspect ratio was temporarily hard-coded to 1 while the browser viewport remained wide. This caused visible horizontal distortion because the projection matrix used a square aspect ratio that did not match the viewport. The original aspect-ratio calculation was restored afterward.





E5 — Frame-rate independence. During a five-second sample, the mean Δt was 0.0171 s and the maximum was 0.0501 s. With 4× CPU throttling, one orbit still took approximately 15 seconds. After I temporarily removed the Δt clamp and switched tabs for about five seconds, the scene jumped forward when I returned; the status showed $t = 5.8$. This happened because the next frame used the full elapsed time. Restoring the 0.1-second clamp limits that jump.



E6 — One vertex by hand. At $t = 1.745$ s, the cube's rotation angle is approximately 120° around the normalized axis $(1, 1, 1)$. The corresponding rotation matrix is approximately $M = \begin{bmatrix} 0 & 0 & 1 & 0 \\ 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$. For $v = (0.5, 0.5, -0.5, 1)$, hand multiplication gives $M \cdot v \approx (-0.5, 0.5, 0.5, 1)$, which matches the `vec4.transformMat4` result. The clip-space output is approximately

$(-0.58864, 0.79771, 7.36015, 7.48592)$. Dividing each of the first three coordinates by $w = 7.48592$ gives $NDC \approx (-0.07863, 0.10656, 0.98320)$.

Development log

After adding the E5 timing logger, the scene rendered as a black screen. I found that the timing variables were declared inside render after they were used. I moved their declarations outside render and confirmed that the scene rendered correctly. Commit: 1d491e5